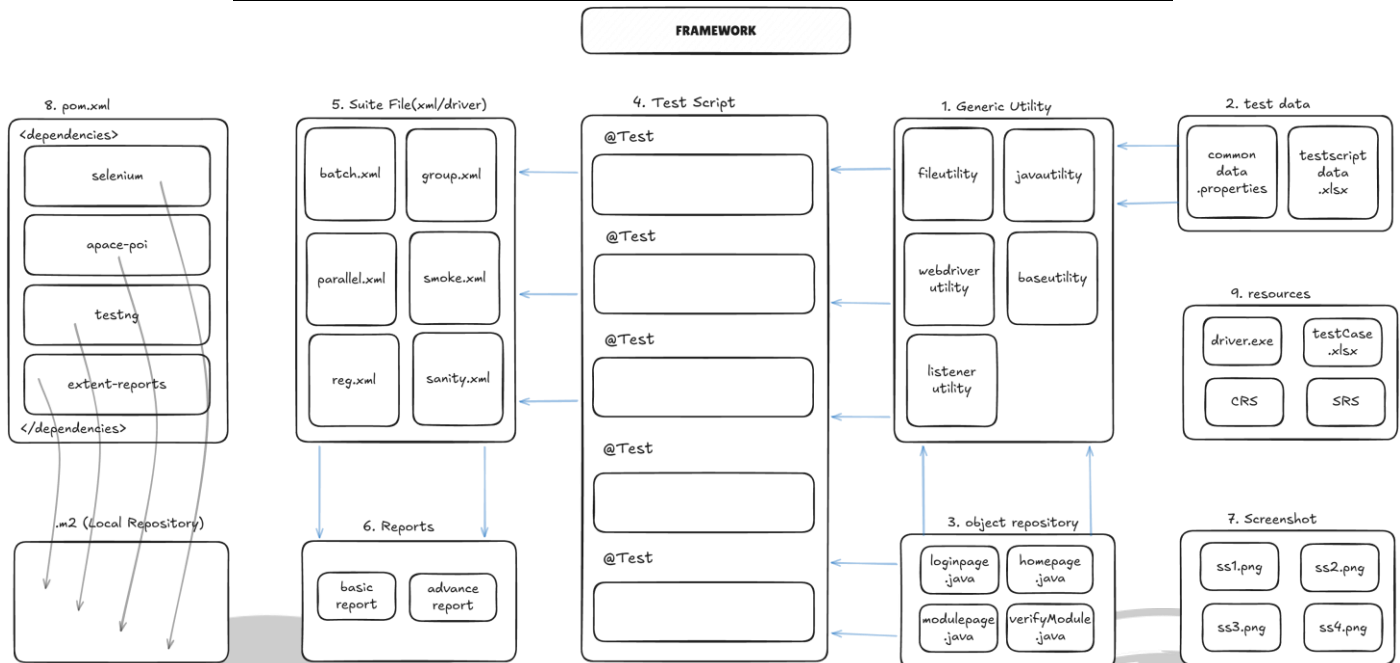


FRAMEWORK EXPLANATION



1. Generic Utility.
2. Test data
3. Object Repository
4. Test Scripts.
5. Test Suite
6. HTML Report.
7. Screenshots.
8. pom.xml
9. Resources

Definition

- **Framework:** A framework is a well-organized structure of reusable components where one driver file will take care of the execution without any manual intervention.

“In my project, I have worked on a **hybrid Selenium automation framework using Java, Selenium, TestNG, Maven, and Page Object Model**.

The main objective of the framework was to make the automation **reusable, maintainable, scalable, and easy to integrate with CI/CD**.

Let me explain the framework structure and execution flow.

1. Generic Utility

First, we have the **Generic Utility** package.

This package contains all the common and reusable methods that can be used across multiple test cases.

For example, we have a **Base Class** where we manage browser initialization, launching the application, and closing the browser. We use TestNG annotations such as `@BeforeMethod` and `@AfterMethod` for login – logout and `@BeforeClass` and `@AfterClass` for setup and teardown.

We also have a **WebDriver Utility**, where we keep common Selenium operations such as handling waits, alerts, frames, windows, dropdowns, and other browser-related operations.

Apart from that, we have utilities for **Java operations, Properties files, and Excel files**.

The main purpose is reusability. If I have already written a common method once, I don't want to write the same logic again in every test script. I simply call that utility method wherever required.

2. Test Data

The second important component is **Test Data**.

The data which is common for all the test scripts is called as common data, and is stored in JSON file

The data which is common for particular test scripts is called as test script data, and is stored in excel file

3. Object Repository

Next is the **Object Repository**.

For this, we follow the **Page Object Model, or POM** design pattern.

Basically, every important page of the application has its own Java class.

For example, we can have a `LoginPage`, `HomePage`, `CustomerPage`, and `ProductPage`.

Inside these Page Object classes, we maintain the **locators and page-level actions** or business utility for example `login()`

The biggest advantage is maintainability.

4. Test Scripts

After that, we have the **Test Scripts**.

Our actual test scenarios are implemented using **TestNG classes**.

For example, we can have separate test classes for login, customer management, product management, or order management.

We use TestNG features such as **@Test, groups, priorities, dependencies, DataProvider, listeners, and parallel execution**, depending on the requirement.

The test classes mainly contain the **business flow**.

They call methods from the Page Object classes and Generic Utilities instead of hardcoding data and locator, it keeps our test scripts clean and readable.

5. Test Suite

Then we have the **Test Suite**.

We use **TestNG XML files** to control which tests need to be executed.

For example, we can create different suites for:

- Smoke testing
- Sanity testing
- Regression testing
- Cross-browser testing

Instead of manually selecting every test case, we can execute the required test suite through the TestNG XML file.

We can also configure things such as **test classes, groups, browsers, and parallel execution** through the suite configuration.

6. HTML Report

For reporting, we generate **HTML reports** after test execution.

We have 2 types of reports, basic and advance reports, basic reports will be auto generated once the testng is integrated with the framework, and advance report we need to configure using third party tool such as extent report or allure report, I have used extent report library for Advance reporting in my framework.

The report gives us a clear overview of the execution, such as which test cases **passed, failed, or were skipped**.

7. Screenshots

We have also implemented **screenshot capture for failed test cases**.

For this, we can use a **TestNG Listener**, particularly `ITestListener`.

When a test case fails, the listener triggers the screenshot-capturing logic.

The screenshot can then be attached to the report or can be stored in the screenshot folder component

8. pom.xml

Next is the `pom.xml` file.

Since our project is Maven-based, we use `pom.xml` for **dependency management and maven build lifecycle**.

For example, dependencies related to **Selenium, TestNG, Apache POI, reporting libraries**, and other required libraries are maintained in the `pom.xml`.

Maven also helps us execute the project from the command line and makes it easier to integrate our automation framework with **Jenkins for CI/CD**.

9. Resources

Finally, we have our **Resources**.

At the project level, we maintain a separate **Resources folder** for supporting project documentation and manual testing-related files.

For example, we keep:

- **Manual Test Cases in Excel format**
- **SRS — Software Requirements Specification**
- **CRS — depending on the project's documentation terminology**
- **Driver executable files**, such as `.exe` files when required
- Other project-related supporting documents

The reason for maintaining these separately is to keep the automation execution resources separate from the **project documentation and supporting files**.

Overall Framework Flow

The main advantages of our framework are **reusability, maintainability, readability, scalability, and reduced code duplication**.

overall, the framework follows **separation of concerns**, where test logic, page elements, reusable utilities, test data, execution configuration, reporting, and project documentation are maintained separately.

That makes the framework easier to maintain and scale as the application grows.”

Framework Advantages

1. **Saves cost & effort** – Less rework, more return on investment.
2. **Covers more tests** – Increases efficiency and test coverage.
3. **Run on any environment** – Works on Testing, UAT, SIT, etc.
4. **Fast test writing** – Reusable parts make scripts quicker to build.
5. **Easy to maintain** – Data & elements stored in one place (POM + external files).
6. **Test anywhere** – Cross-browser, distributed, and agile-friendly.
7. **Auto screenshots** – Captures failures for quick debugging.
8. **Clear reports** – Gives structured and accurate results.
9. **Handy utilities** – Excel, Base class, Listeners, etc. ready to use.
10. **Reusable scripts** – Same test can work across builds.

Types of Frameworks

1. Data driven Framework
2. Modular driven framework

3. Test driven development framework
4. Hybrid Driven framework
5. Keyword Driven framework
6. Behaviour driven development framework
7. (BDD)

1. Data-Driven Framework (DDF)

- **Includes 9 components:**
 1. Generic Utility
 2. Object Repository
 3. Test Data
 4. Test Script
 5. Resources
 6. Suite File
 7. Screenshot
 8. HTML Reports
 9. pom.xml

Key Features

- Simplifies test data maintenance.
- Uses `@DataProvider` annotation.
- One test script can run with multiple data sets.
- Each test script has its own Data-Provider or Excel sheet.

When to Use?

- When large amounts of data need to be tested.
- Examples: E-commerce, Banking, Booking applications.

Pros

- Reduces the number of test scripts.
- Allows testing multiple scenarios with minimal code.

Cons

- Requires skilled testers, proficient in programming to design and maintain the framework.

2. Modular-Driven Framework (MDF)

When to Use?

- For large applications with multiple modules (e.g., CRM systems).

Key Features

- Application is divided into small modules.
- Test scripts are created for each module independently.

Pros

- Easier to manage and scale.
- Independent test scripts for each module.

Cons

- Maintaining consolidated reports for scripts is time-consuming.
- Not preferred in modern agile terms.

3. Keyword-Driven Framework (KDF)

When to Use?

- When manual testers need to write automation scripts with minimal coding knowledge.
- Best for small applications (below 100 test cases).

Key Features

- Test cases are written in Excel.
- Keywords map to key generic libraries (e.g., `openBrowser()`, `ClickElement()`, `EnterText()`).
- Not Suitable for **large** CRM-type applications bcz it **slow down** execution for **thousands of test cases**.

TC_ID	StepDescription	Keyword	LocatorType	LocatorValue	TestData
TC_01	Open URL	OpenBrowser()	URL	https://example.com	
TC_01	Enter username	EnterText()	id	username	John
TC_01	Enter password	EnterText()	id	password	12345
TC_01	Click login button	ClickElement()	xpath	//button[@id='login']	
TC_01	Close browser	CloseBrowser()			

```
public class Keywords {
```

```
    WebDriver driver;
```

```
    public void openBrowser(String url) {
```

```
        driver = new ChromeDriver();
```

```
        driver.manage().window().maximize();
```

```

    driver.get(url);
}

public void enterText(String locatorType, String locatorValue, String testData) {
    getElement(locatorType, locatorValue).sendKeys(testData);
}

public void clickElement(String locatorType, String locatorValue) {
    getElement(locatorType, locatorValue).click();
}

public void closeBrowser() {
    driver.quit();
}

private WebElement getElement(String locatorType, String locatorValue) {
    switch(locatorType.toLowerCase()) {
        case "id": return driver.findElement(By.id(locatorValue));
        case "name": return driver.findElement(By.name(locatorValue));
        case "xpath": return driver.findElement(By.xpath(locatorValue));
        case "css": return driver.findElement(By.cssSelector(locatorValue));
        default: throw new RuntimeException("Invalid locator type: " + locatorType);
    }
}
}
}

```

```

public class TestScript {

```

```

    public static void main(String[] args) throws Exception {

        FileInputStream fis = new FileInputStream("TestSteps.xlsx");

        Workbook workbook = WorkbookFactory.create(fis);
    }
}

```

```
Sheet sheet = workbook.getSheet("sheetName");

Keywords keywords = new Keywords();

for (int i = 1; i <= sheet.getLastRowNum(); i++) {

    Row row = sheet.getRow(i);

    String keyword = row.getCell(2).getStringCellValue();

    String locatorType = row.getCell(3).getStringCellValue();

    String locatorValue = row.getCell(4).getStringCellValue();

    String testData = row.getCell(5).getStringCellValue();

    switch(keyword) {

        case "OpenBrowser": keywords.openBrowser(locatorValue); break;

        case "EnterText": keywords.enterText(locatorType, locatorValue, testData); break;

        case "ClickElement": keywords.clickElement(locatorType, locatorValue); break;

        case "CloseBrowser": keywords.closeBrowser(); break;

        default: System.out.println("Invalid keyword: " + keyword);

    }

}

workbook.close();

}
```

Pros

- Easy maintenance and reusability.
- Non-coders can contribute via Excel.
- Makes automation highly modular and flexible.
- Clear separation between **test logic**, **test data**, and **test script**.

Cons

- **Slower execution** due to reading from external files.
- **Harder debugging** if keywords or mapping logic fails.
- Requires good **framework design discipline**, else it becomes a mess of keywords.

- Poor performance with **large datasets**.

Framework Development Stages

Automation frameworks are not built all at once—they go through **3 main stages**. Think of it like building a house: first you plan, then you build, and finally you live in it and use it!

1. Framework Design & Implementation (Planning and Setup Phase)

When? – Usually done in **Sprint 1** (the first few days/weeks of the project)

Who? – Experienced Automation Test Engineers (7 – 9 years)

What happens here?

- Decide **which tools** to use (like Selenium, TestNG, Maven, Jenkins).
- Create basic **utility classes** (e.g., for browser handling, data reading).
- Set up the **project structure** and folders.
- Create a small working model (called **POC – Proof of Concept**) for important features.
- Show this POC to the client or manager to get their approval.

Goal: Build the base or skeleton of the framework.

2. Framework Test Development (Building Phase)

When? – Usually in **Sprint 2** and continues through development

Who? – Automation Test Engineers (3 – 5 years)

What happens here?

- Start writing **real automation scripts** using the framework.
- Convert **manual test cases** into automation code.
- Store and organize all scripts in version control tools like **GitHub**.
- Review the code with teammates or seniors to ensure quality.

Goal: Fill your framework with working, reviewed, and reusable test scripts.

3. Framework Execution (Using and Monitoring Phase)

When? – Happens regularly after scripts are ready (during every release or sprint)

Who? – Mostly Senior Automation Test Engineers or DevOps team

What happens here?

- Test scripts are run using **Jenkins** or other CI tools (can be scheduled).
- After execution, an **HTML report** is generated showing pass/fail/skipped.
- If any script fails, screenshots are automatically captured.
- Engineers do **debugging**, **fix issues**, and **retest**.

Goal: Run the tests **smoothly**, find bugs, and provide reports to the team or client.

Summary:

Stage	Name	Main Tasks	Who Does It
1	Design & Implementation	Set up tools, create POC, write base classes	Senior Automation Engineers
2	Test Development	Write and review automation scripts	Automation Engineers
3	Execution	Run tests, report results, fix issues	Senior Testers or DevOps

